

Lab 5 — Model Registry Governance

Track A (tabular fraud-detection) · Week 5 · DS5619 Machine Learning Systems Operations

Setup

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python generate_for_student.py --student-id <your roll number or institute email>
```

This overwrites `data/candidate_a/{model.json,metrics.json}` and `data/candidate_b/{model.json,metrics.json}` with values generated deterministically from your student ID — same structure as everyone else's (candidate_a always fails the 0.70 f1 production bar, candidate_b always clears it — otherwise the governance-gate point wouldn't demonstrate anything), but different actual threshold/metric numbers. Two students never get the same data.

Record your `--student-id` value in `NOTES.md`. The grader re-runs `generate_for_student.py` with the ID you recorded and diffs the result against what you committed, and checks that your `registry_summary.json`'s production metrics actually match YOUR candidate_b, not a generic or shared example.

Learning objective

This week's lecture covered the model registry as an artifact store, model cards as the one-page governance record, and why the registry — not a Slack thread or someone's memory — is the answer to "what's actually in production." You'll build a **minimal local model registry** that enforces those two governance rules as code, not just policy: no promotion to Production without a complete model card, and no promotion without metrics that clear a quality bar.

This week's lecture also named where the problem starts: a hyperparameter search or AutoML run producing many near-identical candidates (like `candidate_a/candidate_b` here, just two of them) — explicitly scoped as model-development work this course doesn't teach as a lab, because the registry is what happens *after* that search ends, which is exactly what you're building.

Files

- `src/mini_model_registry.py` — implement the four `# TODO` functions.
- `src/run_pipeline.py` — complete driver script. Don't edit.
- `model_card_fields.json` — fill in with real content before running the pipeline (it will refuse to run while any `TODO` remains).
- `data/candidate_a/`, `data/candidate_b/` — your two personalized model candidates + their metrics, generated above (don't hand-edit).

Background

`data/candidate_a/` and `data/candidate_b/` are two already-trained candidate models for a fraud-detector (deliberately simple: a single amount threshold), each with its own `metrics.json`. One clears production quality bar, one doesn't — you won't be told which until you run the pipeline and see the registry enforce it.

Your task

Part 1 — `src/mini_model_registry.py` (four functions marked `# TODO`, each has a full docstring spec)

- `register_model(name, model_path, metrics, registry_dir)` — the artifact store: version a model file + its metrics, initial stage `"None"`.
- `generate_model_card(name, version_id, card_fields, registry_dir)` — the governance record: reject anything with a missing or `TODO`-containing field, otherwise write it.
- `promote_model(name, version_id, target_stage, registry_dir)` — the gate: Production requires a card AND `metrics["f1"] >= PRODUCTION_F1_THRESHOLD`; a successful promotion to Production archives whichever version was previously there.
- `get_production_model(name, registry_dir)` — what's actually in production, right now, no memory required.

Part 2 — `model_card_fields.json` (fill in real content)

Replace every `"TODO: ..."` placeholder with a genuine 1-2 sentence answer. `src/run_pipeline.py` refuses to run at all while any placeholder remains — that's the same "must actually be filled in" rule your `generate_model_card` function enforces, applied to you first.

```
python src/run_pipeline.py
```

It registers both candidates, deliberately attempts two promotions that should be blocked (no card, then f1 too low) and prints why each was blocked, then successfully promotes the model that clears the bar and writes `registry_summary.json`.

Self-check

```
pytest tests/ -q
```

This is a self-check, not the grader.

Deliverables (what you commit)

- `src/mini_model_registry.py`, completed.
- `model_card_fields.json`, filled in with real content (no `TODO` left).
- The `.model_registry/` directory your pipeline run produced (small JSON manifests + cards only).
- `registry_summary.json`.

- A short **NOTES.md**: the `--student-id` value you used (required — see above), which candidate ended up in Production and why, what would you need to add to `promote_model`'s gate if you also wanted to block promotion of a model trained on stale (e.g. >30-day-old) feature data, and — tying back to this week's AutoML/HPO framing — if a hyperparameter search had handed you 40 candidates instead of 2, what in your `register_model/promote_model` design would need to change (or genuinely wouldn't) to gate 40 instead of 2?

Grading checklist

- ☐ `data/` matches what `generate_for_student.py --student-id <NOTES.md value>` actually produces.
- ☐ `register_model` correctly versions models and never overwrites a prior version.
- ☐ `generate_model_card` genuinely rejects incomplete/TODO cards (checked against a held-out incomplete card, not just the one you tested with).
- ☐ `promote_model` blocks Production promotion on both governance conditions independently, and correctly archives the prior Production version on a successful promotion.
- ☐ `get_production_model` returns the right version, and `None` when nothing is in Production.
- ☐ `model_card_fields.json` is genuinely filled in — no leftover `TODO`, answers are specific to this model, not generic filler.
- ☐ **NOTES.md** shows real reasoning about the second and third questions, not just a restatement of the first.
- ☐ Meaningful commit history and a working README.

Submission

```
git add -A
git commit -m "Week 5: model registry governance"
git tag week05-submit
git push origin main --tags
```